

LEARNMATE

AI Study Support Platform — Project Documentation

Project detail	Description
Project name	Learnmate
Application type	Web application / REST API (based on the supplied backend code screenshots)
Backend technology	Node.js, Express.js
Database	MongoDB with Mongoose
AI integration	Google Generative AI SDK (Gemini model configured in source)
Authentication	JWT-based authentication; student and admin roles
Team / institution	To be filled by the project team
Document basis	User-provided source-code screenshots and "Sample Project Documentation" PDF

Project Description

Learnmate is presented in the supplied code as an AI-supported study-material application. Its backend provides authentication, material-related APIs, administrative routes, file-upload handling, MongoDB data models, and a Gemini integration. The Material model stores a user reference, title, content, original filename, summary, flashcards, quiz items, and a study-plan field. The exact end-user workflow and frontend screens are not included in the supplied screenshots, so this document describes the backend capabilities visible in the code without assuming additional implemented features.

Scope note: The documentation follows the organization and level of detail of the supplied sample. Some implementation details (complete route logic, frontend, actual API request/response formats, package scripts, and deployment) are not visible. Such items are marked "To be confirmed."

Project Scenarios

The following are illustrative usage scenarios based on the visible models and routes, not verified end-to-end demonstrations:

Scenario	Illustrative flow
1. Student study material	A student signs in and submits supported study material. The backend accepts material-related requests and the Material model can store content and generated learning fields such as summary, flashcards, quiz, and study plan.
2. AI-assisted learning content	A material-processing request can use the Gemini integration to generate learning support. The exact prompt, generation functions, and output validation are not fully visible in the supplied screenshots.
3. Administrator access	A request carrying a valid Bearer JWT is authenticated. The adminOnly middleware checks whether the decoded user role is "admin" before allowing an admin-only operation.

Technical Architecture

The visible backend follows a modular Express architecture. The entry file configures middleware and mounts separate route modules. Route handlers are organized in controllers, data is represented through Mongoose models, and reusable functions are placed in utilities and middleware.

Layer	Visible components	Responsibility supported by screenshots
Client / frontend	Not provided in screenshots	A client is expected to call the API; the configured CORS origin defaults to <code>http://localhost:3000</code> when <code>CLIENT_URL</code> is not set.
API server	<code>index.js</code> , Express, CORS, <code>express.json</code>	Creates the Express app, enables JSON parsing and CORS, mounts API routes, exposes a health endpoint, and starts the server after DB connection.
Routes and controllers	<code>auth</code> , <code>materials</code> , <code>admin routes</code> ; <code>authController</code> , <code>materialController</code> , <code>adminController</code>	Separates API route definitions from request-handling logic. Full endpoint details are not visible.
Authentication and upload middleware	<code>auth.js</code> , <code>upload.js</code>	Verifies JWT Bearer tokens, checks admin role, and configures Multer file uploads.
Data layer	<code>User.js</code> , <code>Material.js</code> , <code>db.js</code>	Uses Mongoose to connect to MongoDB and define user/material schemas.
AI service	<code>gemini.js</code>	Initializes Google Generative AI with an environment-provided API key and selects a Gemini model.

High-level request flow

```
Client / UI (not included in screenshots)
  |
  | HTTP request to /api/...
  v
Express application (index.js) ---- CORS + JSON middleware
  |
  +---- /api/auth      -> auth routes / auth controller
  +---- /api/materials -> materials routes / material controller
  +---- /api/admin     -> admin routes / admin controller
  |
  +---- auth middleware (JWT / role checks)
  +---- upload middleware (Multer; supported file types)
  |
  +---- Mongoose models -> MongoDB
  +---- Gemini utility   -> Google Generative AI service
```

Architecture diagram is a text representation derived from the visible file structure and imports. It does not establish the exact runtime call sequence for every endpoint.

Prerequisites and Technologies

Technology / requirement	Use in this project (visible evidence)
Node.js and npm	Runtime and dependency management for the JavaScript backend; <code>package.json</code> and <code>package-lock.json</code> are present.
Express.js	HTTP server and API routing.
MongoDB / Mongoose	Database connection and schema/model definitions.
JWT (jsonwebtoken)	Bearer-token verification and role-based admin check.
bcryptjs	Password hashing and password comparison in the User model.
Multer	File upload handling with disk storage.
Google Generative AI SDK	Gemini model integration in <code>src/utlils/gemini.js</code> .
Environment variables	Configuration includes <code>MONGO_URI</code> , <code>JWT_ACCESS_SECRET</code> , <code>CLIENT_URL</code> , <code>PORT</code> and a Gemini API key variable; verify exact names in source.

Project Workflow

Milestone	Activities / deliverable
1. Project setup	Prepare Node.js project, install dependencies, configure environment variables, and establish MongoDB connectivity.
2. Backend structure	Organize controllers, middleware, models, routes, and utilities as shown in the project tree.
3. Authentication	Create user schema with name, email, password and role; hash passwords before saving; compare passwords during authentication; verify JWTs on protected routes.
4. Material management	Define the Material schema and expose material routes/controller. The full CRUD behavior is to be confirmed from the complete source.
5. File upload	Configure Multer disk storage, unique filenames, allowed extensions (.txt, .md, .pdf), and a 5 MB file-size limit.
6. AI integration	Initialize Google Generative AI using the environment key and configured model; connect AI processing to the material workflow after verifying controller logic.
7. Admin functions	Mount admin routes and apply admin-only authorization where required; verify exact protected endpoints.
8. Testing and deployment	Run the API, test authentication/material/admin endpoints, verify database and AI connectivity, and document deployment details. Status is to be confirmed.

Milestone 1 — Project setup and server initialization

The visible `index.js` loads `dotenv`, imports `Express`, `CORS`, the database connection utility, and `filesystem/path` modules. It creates the `uploads` directory if it does not exist, enables `express.json()`, configures `CORS` using `CLIENT_URL` (with a `localhost:3000` fallback), and mounts three route groups: `/api/auth`, `/api/materials`, and `/api/admin`. The root route returns a JSON health-check message. The server listens on `PORT`, defaulting to `5000`, after `connectDB()` resolves.

Milestone 2 — Database connection

`src/utils/db.js` imports `Mongoose` and `dotenv`, reads `MONGO_URI`, connects with `mongoose.connect()`, and logs the database host after a successful connection. The connection function is exported for use by the entry point.

Milestone 3 — User model and authentication

The User schema contains `name`, `email`, `password`, and `role`. `Email` is required, unique, and lowercased; `role` is restricted to `student` or `admin` and defaults to `student`. A pre-save hook hashes a modified password with `bcryptjs` using 10 salt rounds. The `matchPassword` method compares a supplied password with the stored hash. The `auth` middleware expects an `Authorization` header beginning with `"Bearer "`, verifies the token using `JWT_ACCESS_SECRET`, assigns the decoded payload to `req.user`, and returns `401` for missing/invalid/expired tokens. The `adminOnly` middleware returns `403` unless `req.user.role` is `"admin"`.

Milestone 4 — Material model and upload handling

The Material schema references a User document and includes required title and content fields, filename, summary, flashcards (question/answer pairs), quiz items (question, options, answer), and studyPlan. Mongoose timestamps are enabled. The screenshots show the schema fields but do not show the complete material controller logic or how each field is populated.

The upload middleware uses Multer disk storage and writes files into uploads/. It creates a unique filename using the current time, a random number, and the original extension. The visible filter accepts .txt, .md, and .pdf files; the configured maximum size is 5 MB. The screenshots do not show the text/PDF extraction implementation.

Milestone 5 — Gemini integration

src/utlis/gemini.js imports GoogleGenerativeAI from @google/generative-ai, initializes the client with an environment variable, and selects the model string "gemini-3.7-flash". The visible askGemini function retries temporary errors, including messages containing 503 or 429, with exponential delays. Confirm the model identifier and environment-variable name against the actual installed SDK and current project configuration before final deployment; this document records the screenshot as provided.

Milestone 6 — API routes and controllers

The entry point mounts route modules at /api/auth, /api/materials, and /api/admin. Corresponding controller files are visible: authController.js, materialController.js, and adminController.js. Route files are auth.js, materials.js, and admin.js. Their endpoint names, HTTP methods, request fields, response formats, and validation behavior cannot be established from the screenshots alone.

Milestone 7 — Frontend and user interface

No frontend source files or screenshots are included in the supplied material. Therefore, the UI framework, pages, form fields, navigation, and user-facing output cannot be documented reliably here. Add the frontend folder and screenshots (if part of the project) to complete this section.

Milestone 8 — Testing and deployment

The visible files include a Postman folder and package files, but the contents of the Postman collection, test scripts, npm scripts, hosting provider, and deployed URL are not shown. Verify the application on a clean environment and record the results before submission.

Development and Run Guide (Draft)

The steps below are based on the visible Node.js entry point. Check package.json for the project's actual scripts and version requirements before using these instructions.

Step	Instruction
1	Install a compatible Node.js version and npm.
2	Open the project root directory containing package.json.
3	Run npm install to install dependencies.
4	Create a local .env file. Set MONGO_URI, JWT_ACCESS_SECRET, CLIENT_URL, PORT (optional; default 5000), and the Gemini API key variable used by gemini.js. Confirm the exact Gemini variable name in the source.
5	Ensure MongoDB is running or that MONGO_URI points to a reachable database.
6	Run the start script defined in package.json. If no start script exists, the visible entry file can be started with node index.js.
7	Check the terminal for the server-start and MongoDB connection messages.
8	Visit http://localhost:5000/ (or the configured PORT) to check the health endpoint. The visible code returns "AI StudyBuddy API is running".
9	Use the supplied Postman collection, if complete, to test /api/auth, /api/materials and /api/admin. Confirm exact endpoints from the route files.
10	Verify upload limits, authentication, admin authorization, MongoDB persistence, and Gemini response handling. Record actual test results.

Environment and security notes

Do not commit or submit the real .env file. Keep database credentials, JWT secrets, and Gemini API keys private. Provide a sanitized .env.example containing placeholder values. The screenshots show password hashing and JWT verification, but a complete security review requires the full application source and runtime tests.

Observed Project Structure

```
Learnmate/
├── .postman/
├── .vscode/
├── Code Files/
│   ├── node_modules/
│   └── src/
│       ├── controllers/
│       │   ├── adminController.js
│       │   ├── authController.js
│       │   └── materialController.js
│       ├── middleware/
│       │   ├── auth.js
│       │   └── upload.js
│       ├── models/
│       │   ├── Material.js
│       │   └── User.js
│       ├── routes/
│       │   ├── admin.js
│       │   ├── auth.js
│       │   └── materials.js
│       ├── utils/
│       │   ├── db.js
│       │   ├── gemini.js
│       │   └── tokens.js
│       └── uploads/
├── .env [local only; exclude from submission]
├── index.js
├── package-lock.json
├── package.json
├── README.md
├── node_modules/
└── postman/ [folder shown]
```

Known Limitations and Items to Complete

No.	Item not established by supplied screenshots	Action before final documentation / demo
1	Complete controller and route behavior	Review all route/controller files and document exact endpoints and request/response formats.
2	Frontend implementation and screenshots	Add UI source/screenshots or state clearly if the project is API-only.
3	Exact package scripts and dependency versions	Verify package.json and package-lock.json.
4	Gemini environment-variable name and model availability	Confirm against the actual source, installed package and current API configuration.
5	Text/PDF extraction and AI processing flow	Verify how uploaded content is extracted and passed to Gemini.
6	Database seed data / sample account	Confirm whether seed scripts or demo credentials exist; do not publish real passwords.
7	Automated tests and test results	Run tests or API checks and add dated results.
8	Deployment details	Fill hosting provider, deployed URL, repository link and demo video if applicable.
9	Error handling and file cleanup	Review complete code and test invalid files, oversized files, database failures and AI errors.

Conclusion

Learnmate's supplied backend code shows a modular Node.js and Express API connected to MongoDB through Mongoose, with JWT-based authentication, student/admin roles, material-related data models, upload middleware, and a Google Generative AI integration. The visible Material schema is designed to store study-support outputs such as summaries, flashcards, quizzes, and a study plan. Completing the documentation requires verifying the full route/controller logic, frontend, dependency scripts, test results, and deployment details.

This document is a source-based project documentation draft prepared from the code screenshots and the sample documentation structure. It should be checked against the complete project before submission.